DS1 HW5 Overview

Spring 2022

Recall

- HW3
  - Implement Paxos protocol as a real, functional system
  - Test the system by running randomized test cases many times
- Drawback:
  - No guarantee that all corner cases will happen
    - (e.g. passing 49 out of 50 test runs...)
    - Due to randomness in concurrent systems (e.g. process scheduling, packet reordering)
- Want:
  - **Deterministic** testing for our concurrent protocol
  - Make sure that the procol behaves correctly in **all cases**

HW5: Model Checking Paxos

- Model the entire distributed system as a **state machine**
  - in particular, as a non-deterministic finite automaton (NFA)
- **Events** in the system (e.g. messages, timers) transitions one state to another state
  - since there are multiple possible events, a state can have multiple child states
  - hence, the system non-deterministic progression can be modeled as a state tree
- Advantages:
  - Can explore **every** theoretically possible state
  - Search algorithms (e.g. BFS, DFS) can be used to walk through all the states and verify the correctness of one's implementation

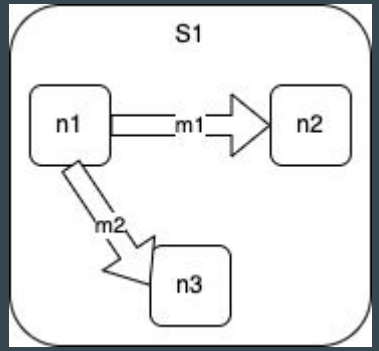
Model: state, event

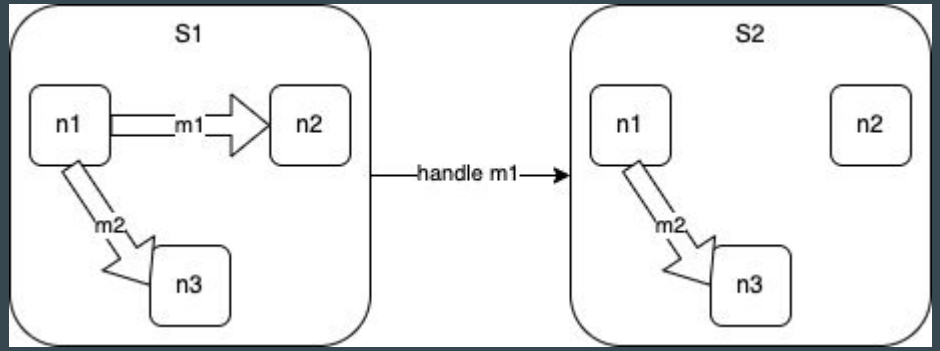
State:

- Nodes + network
- Nodes:
  - can send/receive messages
  - can trigger a timer
- Network:
  - in-flight messages from one node to another

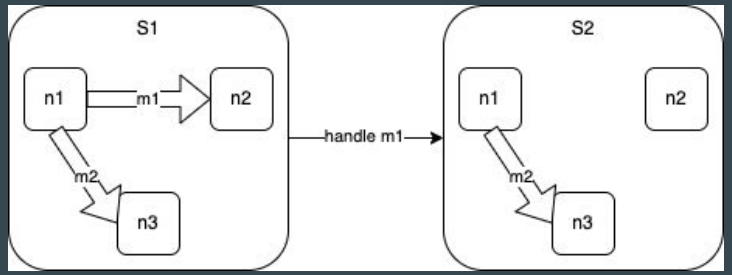
Event:

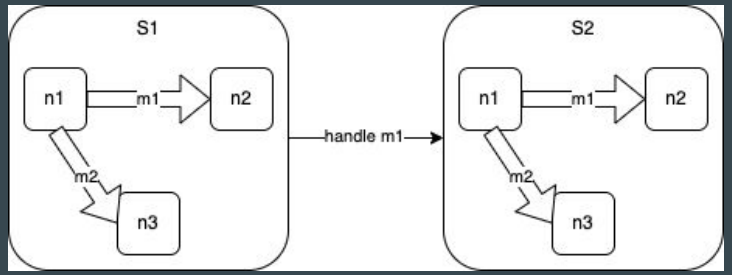
- A node receives a message
- A node's timer is triggered

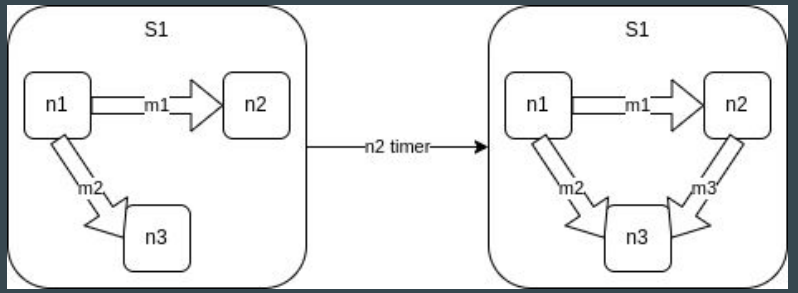




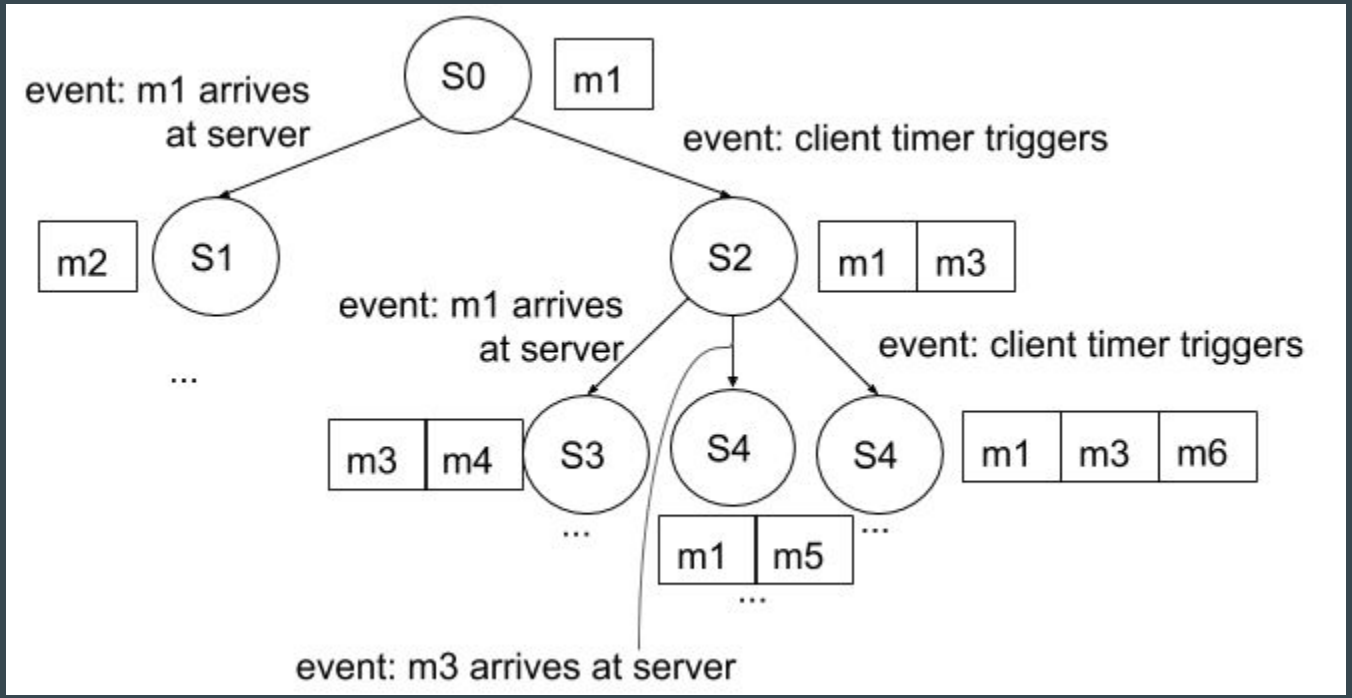
Model: many possible events







Model: non-deterministic state machine



Model: code

Part A

- pkg/base
- Implement the abstraction of state machines
- Important files:
  - node.go
  - message.go
  - timer.go
  - state.go (TODO)

Part B

- pkg/paxos
- Implement Paxos as a state machine, using the base package
- Important files:
  - message.go
  - server.go (TODO)

base/state.go: NextStates()

- Consider all possible events
- Return all states that result from these events
- Should implement and use the following:

```go
func (s *State) HandleMessage(index int, deleteMessage bool) (result []*State) {
	//TODO: implement it
	panic("implement me")
}
```

```go
func (s *State) TriggerNodeTimer(address Address, node Node) []*State {
	//TODO: implement it
	panic("implement me")

}
```

Note: these should probably call HandleMessage() and TriggerTimer() defined for nodes.

```go
func (s *State) NextStates() []*State {
	nextStates := make([]*State, 0, 4)

	for i := range s.Network {
		// check if it is a local call
		if s.isLocalCall(i) {
			newStates := s.HandleMessage(i, true)
			nextStates = append(nextStates, newStates...)
			continue
		}

		// check Network Partition
		reachable, newState := s.isMessageReachable(i)
		if !reachable {
			nextStates = append(nextStates, newState)
			continue
		}

		// TODO: Drop off a message
		if s.isDropOff {
		}

		// TODO: Message arrives Normally. (use HandleMessage)

		// TODO: Message arrives but the message is duplicated. The same message may come later again
		// (use HandleMessage)
		if s.isDuplicate {

		}

	}

	// You must iterate through the addresses, because every iteration on map is random...
	// Weird feature in Go
	for _, address := range s.addresses {
		node := s.nodes[address]

		//TODO: call the timer (use TriggerNodeTimer)
	}

	return nextStates
}
```

base/state.go: RandomNextState()

- Randomly returns 1 single state out of all possible next-states
- Still need to consider all possible transitions

```
func (s *State) RandomNextState() *State {
        timerAddresses := make([]Address, 0, len(s.nodes))
        for addr, node := range s.nodes {
                if IsNil(node.NextTimer()) {
                        continue
                }
                timerAddresses = append(timerAddresses, addr)
        }

        roll := rand.Intn(len(s.Network) + len(timerAddresses))

        if roll < len(s.Network) {
                // check Network Partition
                reachable, newState := s.isMessageReachable(roll)
                if !reachable {
                        return newState
                }

                //TODO: handle message and return one state
        }

        // TODO: trigger timer and return one state
        address := timerAddresses[roll-len(s.Network)]
        node := s.nodes[address]

}
```

paxos/server.go

- Implements the Node interface
  - MessageHandler()
  - TriggerTimer()
- TriggerTimer()
  - calls StartPropose() to setup the propose messages that the `subNode` will send into the network
  - use node SetResponse() function to send messages (*look at Hint in Part A*)
- MessageHandler()
  - heart of the Paxos state machine
  - remember to return **all** possible next-nodes

```
func (server *Server) MessageHandler(message base.Message) []base.Node {
        //TODO: implement it
        panic("implement me")
}
```

```
func (server *Server) TriggerTimer() []base.Node {
        if server.timeout == nil {
                return nil
        }

        subNode := server.copy()
        subNode.StartPropose()

        return []base.Node{subNode}
}
```

```
func (server *Server) StartPropose() {
        //TODO: implement it
        panic("implement me")
}
```

Model Checker: base/search.go

- Predicate:
  - func (*State) bool
- Invariant predicate (validate)
  - a property that should be true for every state in the state tree
- Goal predicate (goalPredicate)
  - a property that is true for the state that we are looking for
- Example:
  - BfsFind()

```
func BfsFind(initState *State, validate, goalPredicate func(*State) bool, limitDepth int) (result SearchResult) {
	queue := list.New()
	queue.PushBack(initState)

	explored := stateHashMap{}
	explored.add(initState)

	result.N = 0

	for queue.Len() > 0 {
		v := queue.Remove(queue.Front())
		state := v.(*State)
		result.N++

		if goalPredicate(state) {
			result.Success = true
			result.Targets = []*State{state}
			return
		}

		// No need to add more states into the queue if the depth is too large
		if limitDepth >= 0 && state.Depth == limitDepth {
			continue
		}

		for _, newState := range state.NextStates() {
			if explored.has(newState) {
				continue
			}

			if !validate(newState) {
				result.Success = false
				result.Invalidate = newState
				return
			}

			explored.add(newState)
			queue.PushBack(newState)
		}
	}

	result.Success = false
	return
}
```

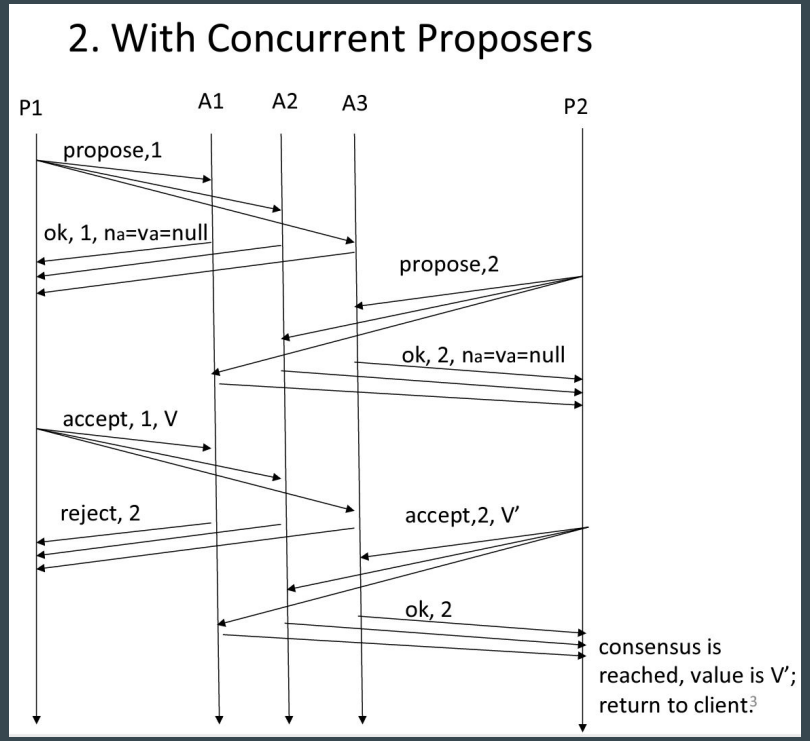
Simple Checks: paxos/scenario_test.go

```go
func validate(state *base.State) bool {
	flag, v := globalAgreedValue(state)

	if flag == -1 {
		return false
	}

	old := state.Prev

	if old != nil {
		_, oldV := globalAgreedValue(old)

		if !base.IsNil(oldV) && oldV != v {
			return false
		}
	}

	return true
}
```

```go
func goal(state *base.State) bool {
	for _, node := range state.Nodes() {
		server := node.(*Server)
		if !base.IsNil(server.agreedValue) {
			return true
		}
	}

	return false
}
```

```go
func TestBfs1(t *testing.T) {
	peers := []base.Address{"s1", "s2", "s3"}

	s := base.NewState(0, false, false)

	server := NewServer(peers, 0, "v1")
	s.AddNode(peers[0], server, nil)

	server = NewServer(peers, 1, nil)
	s.AddNode(peers[1], server, nil)

	server = NewServer(peers, 2, nil)
	s.AddNode(peers[2], server, nil)

	result := base.BfsFind(s, validate, goal, 10)

	if !result.Success {
		t.Fail()
		return
	}

	_, path := base.FindPath(result.Targets[0])
	base.PrintPath(path)
}
```

Scenario Checks



Scenario Checks: paxos/scenario_test.go

```go
func TestConcurrentProposer(t *testing.T) {
	fmt.Printf("Test: Concurrent proposers...\n")
	peers := []base.Address{"s1", "s2", "s3"}
	s := base.NewState(0, false, false)
	server := NewServer(peers, 0, "v1")
	s.AddNode(peers[0], server, nil)

	server = NewServer(peers, 1, nil)
	s.AddNode(peers[1], server, nil)

	server = NewServer(peers, 2, "v3")
	s.AddNode(peers[2], server, nil)

	if nil == reachState(s, concurrentProposerChecks(), 5) {
		t.Fatal("cannot find a path where we reach consensus on v3")
	}
	fmt.Printf("... Passed\n")
}
```

```go
func concurrentProposerChecks() []func(s *base.State) bool {
	p1GetsAllRejects := func(s *base.State) bool {
		if !noConsensus(s) {
			return false
		}
		s1 := s.Nodes()["s1"].(*Server)
		valid := s1.proposer.Phase == Accept && s1.proposer.ResponseCount == 3 && s1.proposer.SuccessCount == 0
		if valid {
			fmt.Println("... p1 received rejects from all peers during Accept phase")
		}
		return valid
	}

	allKnowConsensus := func(s *base.State) bool {
		s1 := s.Nodes()["s1"].(*Server)
		s2 := s.Nodes()["s2"].(*Server)
		s3 := s.Nodes()["s3"].(*Server)
		valid := s1.agreedValue == "v3" && s2.agreedValue == "v3" && s3.agreedValue == "v3"
		if valid {
			fmt.Println("... all peers agreed on v3")
		}
		return valid
	}

	checks := concurrentProposer1()
	checks = append(checks, p1GetsAllRejects)
	checks = append(checks, concurrentProposer2()...)
	checks = append(checks, allKnowConsensus)
	return checks
}
```

Scenario Checks: reachState()

```
func reachState(start *base.State,
        checks []func(s *base.State) bool,
        depthLimit int) *base.State {

        s := start
        for _, check := range checks {

                res := base.BfsFind(s, validate, check, s.Depth+depthLimit)
                if !res.Success {
                        return nil
                }
                s = res.Targets[0]
        }

        return s
}
```

Model Checker: Paxos

- Part C
  - paxos/test_student.go
  - Implement the predicates so that the scenario checks in scenario_tests.go are correct

```
// Fill in the function to lead the program to a state where A2 rejects the Accept Request of P1
func ToA2RejectP1() []func(s *base.State) bool {
        panic("fill me in")
}

// Fill in the function to lead the program to a state where a consensus is reached in Server 3.
func ToConsensusCase5() []func(s *base.State) bool {
        panic("fill me in")
}

// Fill in the function to lead the program to a state where all the Accept Requests of P1 are rejected
func NotTerminate1() []func(s *base.State) bool {
        panic("fill me in")
}

// Fill in the function to lead the program to a state where all the Accept Requests of P3 are rejected
func NotTerminate2() []func(s *base.State) bool {
        panic("fill me in")
}

// Fill in the function to lead the program to a state where all the Accept Requests of P1 are rejected again.
func NotTerminate3() []func(s *base.State) bool {
        panic("fill me in")
}

// Fill in the function to lead the program to make P1 propose first, then P3 proposes, but P1 get rejects in
// Accept phase
func concurrentProposer1() []func(s *base.State) bool {
        panic("fill me in")
}

// Fill in the function to lead the program continue  P3's proposal  and reaches consensus at the value of "v3".
func concurrentProposer2() []func(s *base.State) bool {
        panic("fill me in")
}
```